chapter 04: GPU Memory Hierarchy

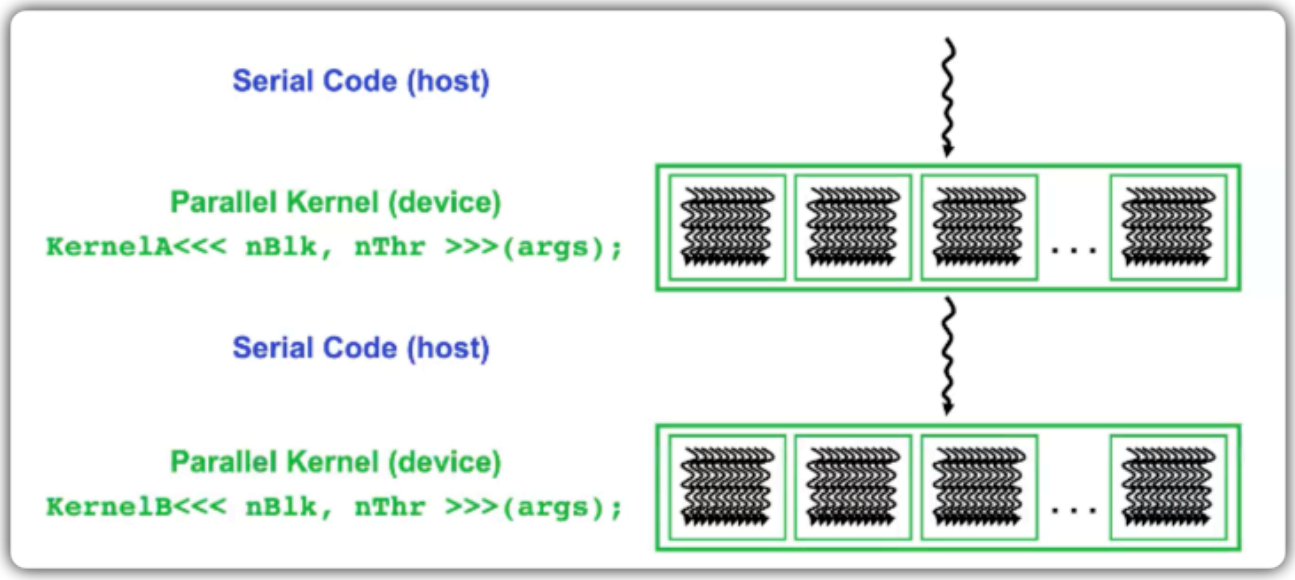
Understanding the **GPU memory hierarchy** is essential for optimizing GPU programs, as this lecture delves into the various **memory spaces** available for threads within a GPU. Viewers will learn how to efficiently utilize resources such as **shared memory** and **global memory** for enhanced performance, particularly in tasks like **matrix multiplication** . The discussion also highlights recent advancements in Nvidia's **H100 architecture** , introducing novel features that allow for improved data communication and processing efficiency. By mastering these concepts, readers will be well-equipped to implement **data reuse** strategies and achieve significant speed-ups in their GPU programming . This lecture is crucial for anyone looking to deepen their knowledge of **GPU programming** techniques.

1. 📊 Introduction to GPU Memory Hierarchy

- This lecture continues the discussion on **GPU programming** , focusing specifically on the GPU memory hierarchy .
- Various **memory spaces** are available for threads when programming a GPU, which will be explored in detail.
- **CPU threads handle sequential or modestly parallel sections, while GPUs execute massively parallel sections.**

2. 🚀 Execution Configuration in GPU Kernel Launch

- Execution configuration defines the **number of blocks per grid** and the number of **threads per block** on a GPU.
- Upon kernel termination, control returns to the host, which may execute serial or parallel code before potentially launching another kernel.



3. 🧩 Vector Addition and GPU Memory Management

🔗

- The vector addition example outlines the process of first **allocating GPU memory** and then transferring data from CPU to GPU memory.
- Computations are executed on the GPU after defining the execution configuration , such as setting the **number of threads per block** .
- In this case, 512 threads per block and the total number of blocks are specified before launching the kernel.
- After kernel execution, results are transferred from GPU back to CPU memory, and then GPU memory is freed or deallocated.

4. 🧩 Advancements in GPU Software Hierarchy with Nvidia H100

🔗

- The **GPU software hierarchy** begins with threads at the lowest level, grouped into thread blocks mapped onto GPU cores .
- A novel feature in the Nvidia H100 architecture introduces **thread block clusters** between thread blocks and the grid.
- This new hierarchy consists of threads, thread blocks, thread block clusters, and finally, the grid.

In parallel computing, especially in GPU programming (e.g., using CUDA), **threads**, **thread blocks**, and **thread block clusters** (or grids) are fundamental concepts that organize computation. Here's a breakdown of each term and a comparison between them:

---

1. Thread

- **Definition**: A thread is the smallest unit of execution in parallel computing. Each thread executes a specific portion of the program, typically working on individual elements of data.
- **Key Characteristics**:
  - Threads are lightweight and run independently.
  - Each thread has its own unique identifier (`threadIdx` in CUDA).
  - Threads share data within a thread block through shared memory.
- **Analogy**: Think of a thread as a worker processing a single piece of data.

---

2. Thread Block

- **Definition**: A thread block is a group of threads that execute concurrently and can share memory. The threads in a block are managed together and can synchronize with each other.
- **Key Characteristics**:
  - A block contains a **fixed number of threads**, often defined as a 1D, 2D, or 3D structure (e.g., `blockDim.x` , `blockDim.y` , `blockDim.z` in CUDA).
  - Threads in a block can communicate via **shared memory** (fast, on-chip memory) and synchronize using barriers.
  - Thread blocks are identified by a unique block ID (`blockIdx` in CUDA).
- **Analogy**: Think of a thread block as a team of workers collaborating on a specific part of a project.

---

3. Thread Block Cluster (Grid)

- **Definition**: A thread block cluster, also known as a grid, is an arrangement of multiple thread blocks. It represents the overall structure of execution at a higher level.
- **Key Characteristics**:
  - A grid contains **multiple thread blocks**, organized in 1D, 2D, or 3D (e.g., `gridDim.x` , `gridDim.y` , `gridDim.z` in CUDA).
  - Threads in different blocks cannot communicate directly and cannot share memory.

- Each thread has a unique global identifier, calculated using `blockIdx` and `threadIdx`.
- **Analogy**: Think of the grid as the entire workforce, with each block working on a part of a larger project.

Comparison Table

Feature	Thread	Thread Block	Thread Block Cluster (Grid)
Definition	Smallest unit of execution	Group of threads	Group of thread blocks
Size/Structure	Single thread	Configured in 1D, 2D, or 3D	Configured in 1D, 2D, or 3D
Identifier	`threadIdx`	`blockIdx` + `threadIdx` for threads	Calculated using `gridDim` and `blockIdx`
Memory Access	Registers, shared memory	Shared memory among threads	Global memory only
Synchronization	No synchronization needed	Possible within the block	Not possible across blocks
Communication	None	Shared memory	Global memory
Scope of Work	Processes a single data item	Processes a subset of data	Processes the entire dataset

---

Key Relationships

1. **Threads** are the building blocks of computation.
2. **Thread blocks** group threads together for localized computation and memory sharing.
3. **Thread block clusters (grids)** combine multiple blocks for large-scale parallelism.

Example in CUDA:

If we want to process a 2D image:

- Each **thread** could process a single pixel.
- A **thread block** could process a tile (e.g., 16x16 pixels).
- A **grid** could encompass all the tiles, covering the entire image.

Understanding these hierarchies helps you divide and conquer problems efficiently on parallel computing platforms!



- Thread block clusters are made up of thread blocks, enhancing the structure of the grid.

    - **Thread blocks in clusters**
        - Thread blocks in the same cluster can synchronize and exchange data
        - Thread blocks in the same cluster are guaranteed to be concurrently scheduled
        - Thread blocks in the same cluster run on the SMs within a GPU Processing Cluster (GPC)
        - Data sharing via SM-to-SM network in a GPC
- In the H100 architecture, threads within these clusters can directly communicate through a new network.

---

5. 🧠 Understanding GPU Memory Spaces and Efficient Data Management

---

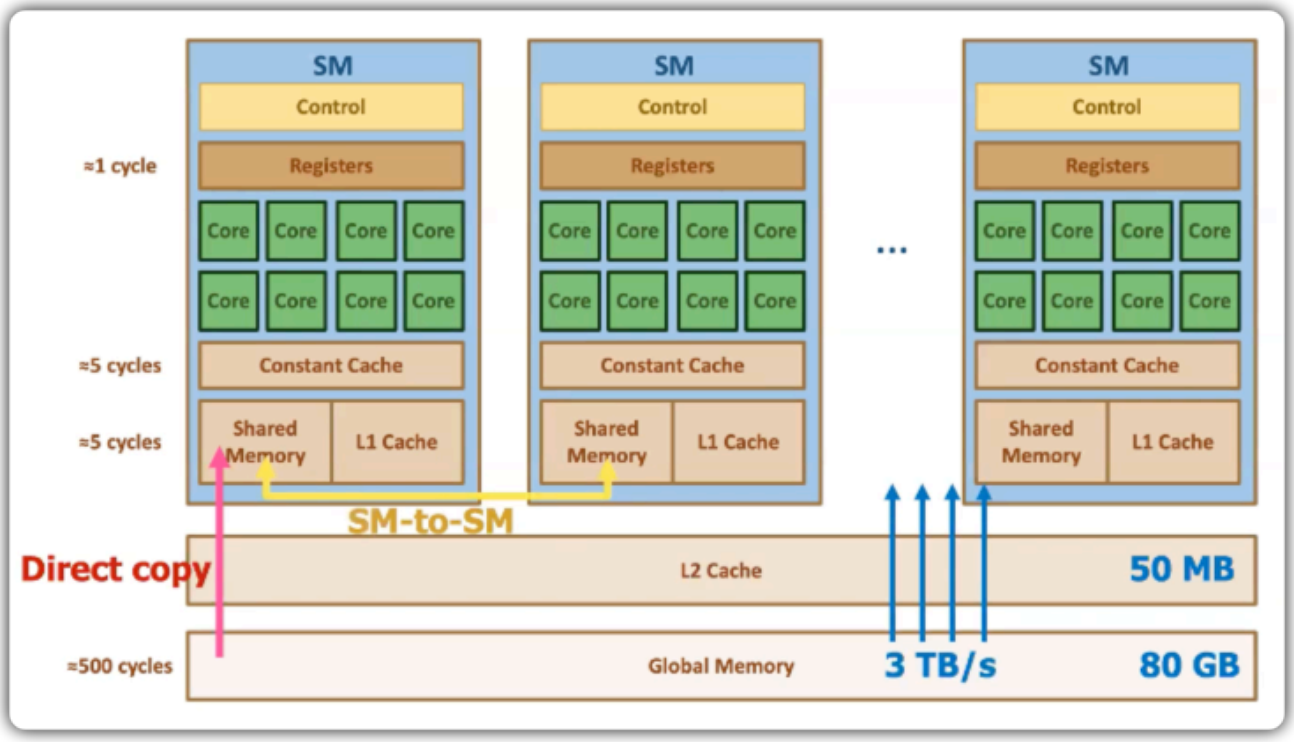
🔗

- Nvidia's H100 GPU architecture includes various **memory spaces** such as registers, shared memory , L1, and L2 caches, and global memory , each with distinct access speeds.
- Shared memory acts as a software-managed scratchpad for threads in the same block, facilitating the reuse of data like in matrix multiplication , enhancing performance by reducing global memory accesses.
- Utilizing **sync threads** is crucial for synchronizing thread operations, particularly when multiple threads access shared memory to avoid data hazards.
- The H100 GPU introduces direct data transfer capabilities from global memory to shared memory and between SMs, improving data handling efficiency.
- Techniques like **tiling** optimize data reuse by loading data tiles into shared memory , significantly accelerating computations compared to accessing global memory .

5.1. NVIDIA H100 GPU Memory Spaces and Features

🔗

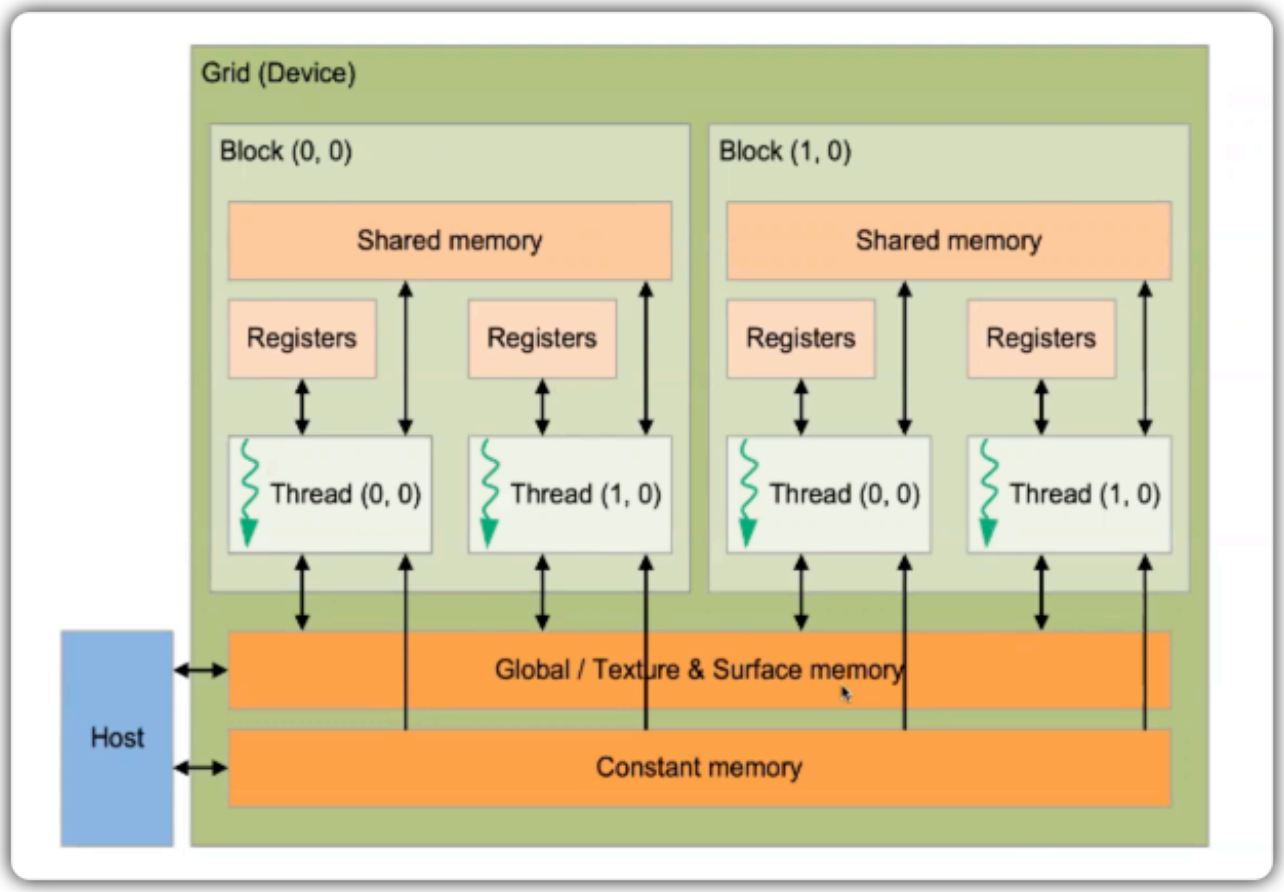
- NVIDIA's H100 GPU integrates various memory spaces, including **registers, L1 cache** , and shared memory , inside its cores.
- **Shared memory** in the GPU, known as scratchpad memory, allows threads within the same block to share data efficiently.
- The access latency to shared memory and L1 cache is about five cycles, while global memory access latency can reach up to 500 cycles.
- H100 GPU features innovations like direct copying from global memory to shared memory and communication from SM to SM within the same cluster.
- H100 offers 80 GB of **HBM3 global memory** with a groundbreaking bandwidth of 3 TB/s, double that of the previous A100 GPU.



5.2. Understanding GPU Memory Allocation

- Different **memory spaces** in GPUs include registers, constant cache , shared memory , L1 and L2 caches, and global memory .
- **Qualifiers** determine the allocation of specific values or arrays in GPU programming .
- Automatic variables are allocated in registers by the compiler, but if registers are insufficient, they utilize off-chip memory.
- Arrays within a GPU kernel are private to threads, and if registers are available, they are allocated there.
- Threads use shared memory within the same thread block and global memory for cross-thread block data access.

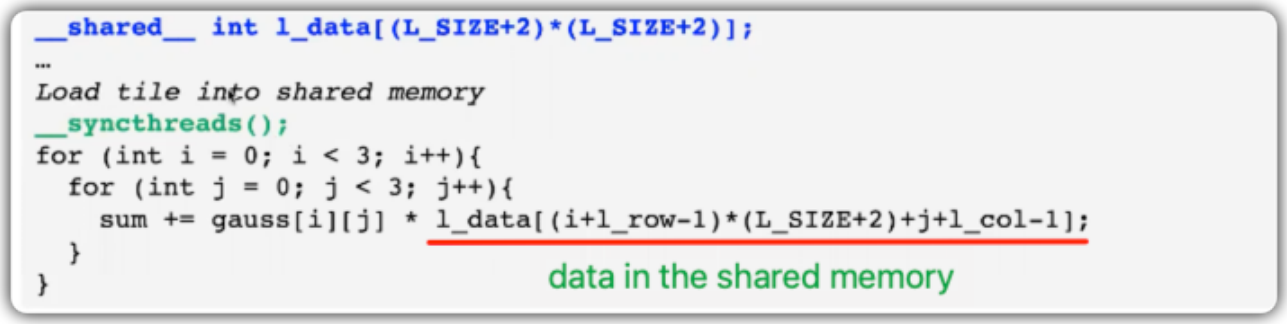
Variable declaration		Memory	Scope	Lifetime
	`int LocalVar;`	register	thread	thread
	`int localArr[N];`	global	thread	thread
`__device__ __shared__`	`int SharedVar;`	shared	block	block
`__device__`	`int GlobalVar;`	global	grid	application
`__device__ __constant__`	`int ConstantVar;`	constant	grid	application



5.3. Efficient Data Reuse in Shared Memory for GPU Programming

- **Shared memory** is advantageous for data reuse within the same thread block , enabling effective communication and data sharing among threads.
- In operations like **Gaussian filtering** on images, threads access shared pixels, demonstrating data reuse by utilizing nearby pixel information for calculations.
- A technique called **tiling** (or blocking) involves defining data tiles reused by multiple threads, improving memory efficiency and computation speed.

- **Shared memory allows all threads in a block to access the same data tile**, necessitating synchronization of threads using the **sync threads** instruction to prevent data hazards.



- Ensuring data tiles are fully loaded into shared memory before computation prevents execution issues, especially with multiple warps in a thread block .

5.4. Efficient Matrix Multiplication Using Shared Memory

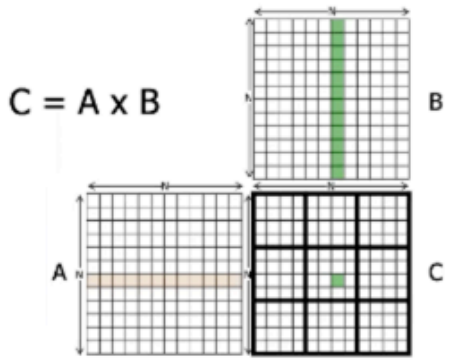
- Matrix multiplication involves computing the product of two matrices, resulting in an output matrix C, with elements formed by dot product calculations from rows and columns of matrices A and B.
- The parallelization of this process is achieved by assigning individual threads to each element of the output matrix C within a GPU, organized into thread blocks.
- Shared memory is leveraged to enhance efficiency by tiling input matrices A and B, loading tiles into shared memory , and then performing partial multiplications.
- Accessing shared memory significantly speeds up calculations compared to accessing global memory , with an improvement of two orders of magnitude.
- The operation is repeated for successive tiles until the final results for matrix C are obtained, utilizing data reuse from shared memory to optimize performance.

```
__global__ void mm_kernel(float* A, float* B, float* C, unsigned int N) {

    unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;

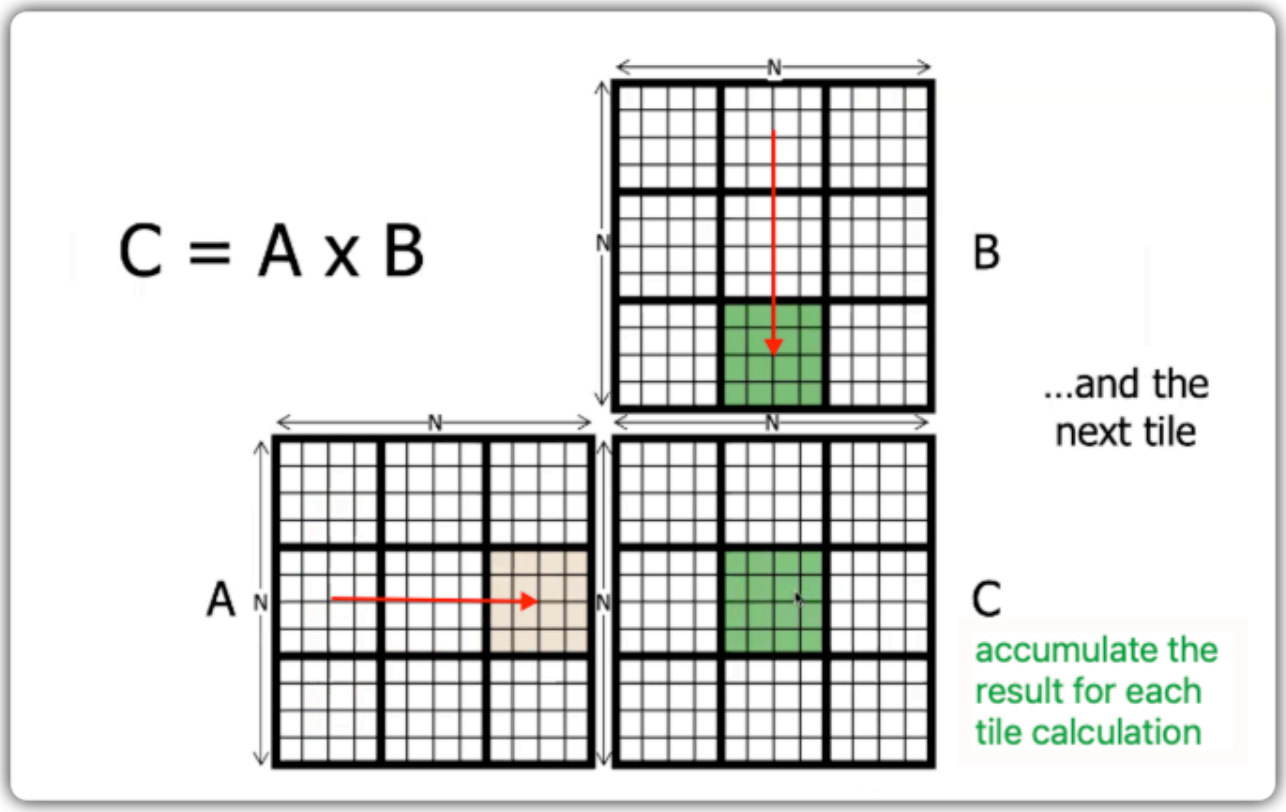
    float sum = 0.0f;
    for(unsigned int i = 0; i < N; ++i) {
        sum += A[row*N + i]*B[i*N + col];
    }
    C[row*N + col] = sum;

}
```



5.5. Optimizing Matrix Multiplication with GPU Programming

- Arrays are declared in **shared memory** , where tiles of matrices A and B are placed for efficient access.
- Each thread initializes an accumulator and reads tiles from **global memory** to shared memory , requiring **synchronization** due to multithreading.
- **After synchronization , threads compute using shared memory tiles and accumulate results in a sum variable for output.**
- Results are written to the final matrix after processing all tiles, ensuring full data usage before replacing tiles.




```
__shared__ float A_s[TILE_DIM][TILE_DIM];
__shared__ float B_s[TILE_DIM][TILE_DIM];

unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;

float sum = 0.0f;

for(unsigned int tile = 0; tile < N/TILE_DIM; ++tile) {

    // Load tile to shared memory
    A_s[threadIdx.y][threadIdx.x] = A[row*N + tile*TILE_DIM + threadIdx.x];
    B_s[threadIdx.y][threadIdx.x] = B[(tile*TILE_DIM + threadIdx.y)*N + col];
    __syncthreads();

    // Compute with tile
    for(unsigned int i = 0; i < TILE_DIM; ++i) {
        sum += A_s[threadIdx.y][i]*B_s[i][threadIdx.x];
    }
    __syncthreads();
}

C[row*N + col] = sum;
```

Declare arrays in shared memory

Threads wait for each other to finish loading before computing

Threads wait for each other to finish computing before loading